claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_b9d6fa25-e41\agent-a9e8f301ea2daa1c5.jsonl`
- SHA-256: `7585432f69ea3a122bf41c9936ff1b67ce0c1768a55285acb295ce9dccd29b4b`
- Source modified: `2026-06-25T17:37:58+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_b9d6fa25-e41`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T17:36:59.044Z)

```
Adversarially verify this PR-review finding for PR #385 (branch b1-p9/remove-fabricated-motion-data checked out; read the code ? STRICT READ-ONLY, do NOT edit any file). Be skeptical ? default to INVALID if wrong, already-handled, or a non-issue.

Finding [MEDIUM] Audit tracker docs/CODE_CLEANUP_AUDIT_2026-06-24.md not updated in this PR (B1/P9 still ?) ? violates the tracked-doc convention #380's review enforced
Location: docs/CODE_CLEANUP_AUDIT_2026-06-24.md (NOT in this PR's diff; git diff origin/master...HEAD touches only motion.py + test_motion_segmenter.py)
Detail: This PR completes B1/P9 (remove fabricated data from the motion segmenter) but does not touch the audit tracker doc at all. The doc is the project's tracked-project-doc per docs/PROJECT_DOC_TEMPLATE.md, and the established, consistently-followed convention is that the PR which lands a tracker item flips its own §6 row to ? with the PR link, removes itself from the §8.1 pick-up list, and adds a dated §changelog entry ? done in the SAME PR (precedents: P8/#373, P11/#380, P17/#378, all visible in the changelog at lines 277-280). Three concrete stale spots remain after this PR merges:
  1. §6 tracker, line 197: `| **P9** | **B1: Remove fabricated data from motion segmenter** | P7 | No | ? | ? |` ? still unchecked, no PR link.
  2. §8.1 'Next items ? Immediate' pick-up list, line 245: B1/P9 is still listed as an open prioritized pick-up (row '3'). Compare the done items 1 and 2 (B2/P17 and O1/P11) at lines 238-239 which were struck through with '? DONE (#NNN)' the moment they landed.
  3. §Changelog (lines 272-280): no `2026-06-25 ? P9 done (B1)` entry, unlike every other completed item.
 This is the exact omission #380's review caught (a completing PR that forgot its tracker edit). CI is green and the code is correct, so this is a convention/process gap, not a code bug ? but it leaves the source-of-truth tracker lying about completion state.
Suggestion: In THIS PR, add docs/CODE_CLEANUP_AUDIT_2026-06-24.md to the diff: (1) flip line 197 to `| **P9** | **B1: Remove fabricated data from motion segmenter** | P7 | No | ? | [#385](...) |`; (2) strike/mark the §8.1 row 245 as `? DONE (#385)` like the other completed pick-up items; (3) append a dated changelog entry (`2026-06-25 ? P9 done (B1). Motion segmenter no longer fabricates server/receiver/shot_count/avg_speed_kmh/events with stdlib random ? passes None/unknown markers matching yolo_hybrid/gemini; all db.add_event calls dropped; random import removed. Suite green.`). Stage the doc path explicitly per the branching-safety convention.

Return isReal + adjustedSeverity (INVALID if not real) + one-paragraph reasoning grounded in the actual code.
```

2. assistant (2026-06-25T17:37:01.721Z)

```
I'll verify this finding by examining the actual code and the PR diff. Let me start by checking the PR's actual diff and the current state of the audit tracker doc.
```

3. user (2026-06-25T17:37:05.622Z)

```
---CURRENT BRANCH---
```

```
b1-p9/remove-fabricated-motion-data
---HEAD---
eabf774 fix(B1/P9): remove fabricated data from motion segmenter
cd5b3d2 feat(O1/P11): Gemini remote-file GC ? startup orphan sweep (#380)
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)
---DIFF FILES vs origin/master---
backend/pipeline/segmenters/motion.py
tests/test_motion_segmenter.py
```

4. assistant (2026-06-25T17:37:08.548Z)

The finding confirms the PR diff only touches `motion.py` and `test_motion_segmenter.py`. Now let me verify the code change is correct and examine the audit tracker doc state.

5. user (2026-06-25T17:37:10.193Z)

```
import logging
import time
from typing import Any, Dict, List, Optional, Tuple

import cv2
import numpy as np

from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
from backend.results import Failure, Result, Success  # standardized contract

logger = logging.getLogger(__name__)

class MotionPlaySegmenter(BasePlaySegmenter):
    @property
    def name(self) -> str:
        return "motion"

    @property
    def description(self) -> str:
        return "Traditional CV pixel-motion heuristics tracking frame differences."

    def process_video(self, video_id: str, video_path: str, player_pool: Optional[List[str]] = None, max_frames: Optional[int] = None) -> "Result[List[Dict[str, Any]]]":
        """
        Processes a video to detect play segments using motion heuristics and
        populates SQLite with detected segments. The ``player_pool`` arg is
        accepted for ABC signature compatibility but no longer used (the motion
        segmenter has no ground truth for server/receiver ? see _populate_db).
        """
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            logger.error(f"Segmenter could not open video: {video_path}")
            return Failure(message=f"Could not open video: {video_path}", is_recoverable=False)

        final_segments = self._detect_motion_segments(cap, max_frames)

        segments_data = self._populate_db(video_id, final_segments)

        return Success(value=segments_data)

    def _detect_motion_segments(self, cap: Any, max_frames: Optional[int] = None) -> List[Tuple[float, float]]:
        """Run the pixel-motion heuristic over ``cap`` and return the detected
        (start, end) play segments.

        Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
        thresholds it into raw segments, then merges dead-time gaps and filters
        out segments shorter than ``min_segment_duration``. ``cap`` is released
```

```
        before returning. This is the detection half of ``process_video`` ? it
        performs no DB writes and no fabrication.

        ``max_frames`` limits the number of analysed frames (Optional; for debugging).
        An explicit ``np.ones((3,3))`` kernel is used for ``cv2.dilate`` rather than
        ``None`` so the typed overload resolves correctly.
        """
        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        duration = total_frames / fps if fps > 0 else 0

        # Sub-sample settings (5 FPS for analysis)
        analysis_fps = 5.0
        frame_interval = max(1, int(fps / analysis_fps))

        idx_cfg = self.config.indexing
        motion_threshold = idx_cfg.motion_threshold
        min_segment_duration = idx_cfg.min_segment_duration
        dead_time_merge_gap = idx_cfg.dead_time_merge_gap

        prev_gray = None
        motion_signals = []
        timestamps = []

        frame_idx = 0
        processed_count = 0
        t_start = time.time()
        # Emit a progress line roughly every 5% of the video so long runs are
        # observable instead of silent (a full 1080p match is ~tens of minutes).
        progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
        next_progress = progress_step
        logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
                    f"(~{duration/60:.1f} min), analyzing every {frame_interval}th frame")
        while True:
            # Skip decoding for intermediate frames using cap.grab() for high performance
            for _ in range(frame_interval - 1):
                if not cap.grab():
                    break
                frame_idx += 1

            ret, frame = cap.read()
            if not ret:
                break

            # Process frame
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            # Apply Gaussian Blur to smooth noise
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            t = frame_idx / fps
            timestamps.append(t)

            if prev_gray is not None:
                # Frame absolute difference (highly optimized motion tracking)
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)

                # Dilate to fill gaps
                kernel = np.ones((3, 3), np.uint8)
                thresh = cv2.dilate(thresh, kernel, iterations=2)

                # Motion ratio
                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
```

```
thresh.shape[1])
                motion_signals.append(motion_ratio)
            else:
                motion_signals.append(0.0)

            prev_gray = gray
            frame_idx += 1
            processed_count += 1

            if progress_step and frame_idx >= next_progress:
                pct = 100.0 * frame_idx / total_frames
                elapsed = time.time() - t_start
                eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
                logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames, "
                            f"{processed_count} analyzed, {elapsed:.0f}s elapsed, ~{eta:.0f}s left")
                next_progress += progress_step

            if max_frames is not None and processed_count >= max_frames:
                logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer frame parsing.")
                break

            if frame_idx >= total_frames:
                break

        cap.release()

        # Smooth signal using a moving average
        window_size = 5
        smoothed = []
        for i in range(len(motion_signals)):
            start = max(0, i - window_size // 2)
            end = min(len(motion_signals), i + window_size // 2 + 1)
            smoothed.append(sum(motion_signals[start:end]) / (end - start))

        # Detect active play boundaries
        in_segment = False
        segment_start: Optional[float] = None
        raw_segments: List[Tuple[float, float]] = []

        for i, t in enumerate(timestamps):
            signal = smoothed[i]
            if signal > motion_threshold:
                if not in_segment:
                    in_segment = True
                    segment_start = t
            else:
                if in_segment:
                    in_segment = False
                    assert segment_start is not None  # set when in_segment became True
                    raw_segments.append((segment_start, t))

        # Handle video ending while in active segment
        if in_segment:
            assert segment_start is not None  # set when in_segment became True
            raw_segments.append((segment_start, timestamps[-1]))

        # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
        merged_segments: List[Tuple[float, float]] = []
        if raw_segments:
            curr_start, curr_end = raw_segments[0]
            for next_start, next_end in raw_segments[1:]:
                if next_start - curr_end < dead_time_merge_gap:
```

```
                    # Merge
                    curr_end = next_end
                else:
                    merged_segments.append((curr_start, curr_end))
                    curr_start, curr_end = next_start, next_end
            merged_segments.append((curr_start, curr_end))

        # Post-Processing: 2. Filter segments shorter than min_segment_duration
        final_segments: List[Tuple[float, float]] = [
            r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
        ]

        return final_segments

    def _populate_db(self, video_id: str, final_segments: List[Tuple[float, float]]
                   ) -> List[Dict[str, Any]]:
        """Persist ``final_segments`` to the DB with honest metadata.

        The motion segmenter detects play segments via pixel-diff heuristics but
        has NO ground truth for who served, shot counts, shuttle speed, or event
        breakdown ? those were previously fabricated with the stdlib ``random``
        module (the legacy demo baseline). They are now persisted as ``None`` /
        unknown markers so consumers can distinguish "not modeled" from a real
        value. Matches yolo_hybrid/gemini, which pass ``server=None, receiver=None``.
        """
        segments_data = []
        for start, end in final_segments:
            dur = end - start
            metadata = {
                "shot_count": None,
                "intensity": "high" if dur > 12 else "medium",
                "avg_speed_kmh": None,
            }
            segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)
            segments_data.append({
                "id": segment_id,
                "start_time": start,
                "end_time": end,
                "duration": dur,
                "server": None,
                "receiver": None,
                "metadata": metadata,
            })
        return segments_data

# Register the segmenter
segmenter_registry.register(MotionPlaySegmenter)

```

6. user (2026-06-25T17:37:10.847Z)

```
<!--
PROJECT DOC ? tracked code-cleanup + audit remediation plan.
Follows the PROJECT_DOC_TEMPLATE.md conventions.
-->

# Code Cleanup ? 2026-06-24 Deep Audit Remediation

> **Status:** `PHASE 1 COMPLETE` ? 14 items shipped + D2 app-factory + test fixes · **Owner:** Avi · **Created:** 2026-06-24 · **Last updated:** 2026-06-25 (DoD/line-count accuracy fixes ? see Changelog)
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/past_projects/` when DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
```

`docs/README.md`.
> **Relation to existing docs:** Sibling of `docs/CODE_AUDIT_AND_TEST_HARDENING.md` (the 2026-06-10 hardening sweep).
>
> **Phase 1 summary:** All structural refactorings landed. `backend.api.job_worker` lifted from mypy quarantine. Suite: **1463/1463 green**. `ruff`: all checks passed. `mypy`: quarantined modules unchanged (ratchet held).

---

0. TL;DR

A deep, whole-repo audit on 2026-06-24 surfaced **35 items** (33 from fresh review + 2 absorbed from `docs/BACKLOG.md`) across bugs, technical debt, operational risks, and quick wins. This doc organizes them into a phased cleanup project: **Phase 1 = pure refactorings** (no logic change, all tests must stay green, coverage must not drop), **Phase 2+ = logic fixes + hardening**. The first approved targets are **B2, D3, D2, B5** ? the `Success`/`Failure` contract for Gemini, typed-config migration completion, app-factory refactor, and the no-op segmenter validator fix.

---

1. Problem & goal

The codebase has grown rapidly since the June 2026 scaffolding and has accumulated several patterns that make it harder to extend safely:

- **Silent failure masking** ? `analyze_video()` returns `[]` on ANY error (missing API key, network failure, model crash) indistinguishable from "video has no rallies."
- **Dual config access** ? new code uses typed `AppConfig` attribute access; old code (validators, segmenters) uses legacy `.get("indexing",{}).get(...)` dict chains. The `extra='allow'` silently keeps typo'd config keys.
- **Optional module globals** ? `db_instance` and `global_config` are `Optional[...] = None`, set only in `main()`. Importing `app` without `main()` ? NPE on every endpoint. The mypy quarantine on `backend.main` and `backend.api.job_worker` exists because of this.
- **No-op validator** ? `segmenter_must_be_registered` field validator is a pass-through; a typo'd segmenter name in `config.json` passes validation and fails at runtime.

The goal is to fix these systematically, with **zero behavior change** in Phase 1, and a **strict no-regression gate** (all tests pass, coverage never decreases).

---

2. Decisions locked

#	Decision	Source / date	Implication
D1	**Phase 1 = pure refactorings only.** No logic or behavior change. All tests must pass; coverage must not decrease.	Owner, 2026-06-24	B2 (Success/Failure contract for Gemini) is deferred to Phase 2 since it changes return types callers depend on. D3 (typed-config migration), D2 (app-factory), and B5 (segmenter validator) are pure-refactoring-possible and are the first targets.
D2	**First targets: D3 ? B5 ? D2** in that order (increasing risk). D3 (typed-config migration) is the safest ? fix the validator callers one module at a time. B5 (segmenter validator) is a one-liner. D2 (app-factory) is the largest but still a refactoring. B2 (Success/Failure in Gemini) is the first Phase-2 item.	Owner, 2026-06-24	Each lands via a separate, small PR branched from `master`; no PR stacking.
D3	**Docs-only PR first.** This document lands as a single PR with no code changes. The subsequent code PRs reference this doc.	Owner, 2026-06-24	The doc is approval for the approach; individual code PRs are approval for the implementation.
D4	**Coverage ratchet:** the CI `coverage` lane must not decrease for any PR in this project.	Convention from `CODE_AUDIT_AND_TEST_HARDENING.md`	Every code PR must include updated/additive tests. Pure refactorings that remove dead code may increase coverage; a drop is a hard fail.
D5	**Follow the CODE_MAP placement rules.** No new top-level files without a

placement-rule match. Tools go in `tools/`, standalone run-drivers in `scripts/`, eval library code in `backend/eval/`. | `docs/CODE_MAP.md` §0 | Keeps the repo navigable as it evolves. |
| **D6** | **mypy quarantine ratchet:** when touching a quarantined module, try removing its `ignore_errors = True` entry from `mypy.ini`. | `mypy.ini` | D2 (app-factory) is the biggest opportunity ? if `backend.main` and `backend.api.job_worker` can leave quarantine, that's a major milestone. |
| **D7** | **Absorb relevant BACKLOG items.** Deferred hardening items from `docs/BACKLOG.md` that overlap with this audit are absorbed into this project rather than left in BACKLOG. Items that are feature-gaps (not code-cleanup) stay in BACKLOG. | PR #352 review, 2026-06-24 | D2 and O1 were already independently found; D13 and D14 are new absorptions. The train/eval splits guardrail stays in BACKLOG (it needs a collector-side manifest change). |

---

3. Findings register (full audit)

3a. Bugs & Defects

ID	Severity	File(s)	Issue	Phase
B1	**HIGH**	`backend/pipeline/segmenters/motion.py`	**Fabricated data.** Random values for server/receiver/shot_count/events/intensity/avg_speed_kmh persisted as real data.	2
B2	**HIGH**	`backend/providers/gemini.py`	**Silent zero-segment returns mask API failures.** `analyze_video()` returns `([], metrics)` on ANY failure, indistinguishable from "no rallies."	2
B3	MEDIUM	`backend/pipeline/validators/pts_continuity.py`	Hardcoded 10.0s gap threshold (comment says 8.0s), not config-driven.	3
B4	MEDIUM	`backend/pipeline/stitcher/compiler.py::_parse_time`	Unparseable timestamps silently default to 0.0 ? could stitch from time zero.	3
B5	MEDIUM	`backend/config/models.py::IndexingConfig`	`segmenter_must_be_registered` validator is a **no-op pass-through**. Typo'd segmenter passes validation.	**1**
B6	LOW	`backend/pipeline/validators/resolution_fps.py`	Hard-fails if `format_check` didn't run first ? ordering is an import-time side-effect.	3
B7	LOW	`backend/eval/calibrate_wasb.py::load_golden_gts`	Silently skips bad rows (opposite of `annotations.load_annotations` which RAISES).	3
B8	LOW	`backend/eval/regression.py`	No baseline ? `regressed=False` (silent pass). Library function diverges from `run_regression.py` exit 3.	3

3b. Technical Debt / Design Smells

ID	Severity	Area	Issue	Phase
D1	HIGH	`backend/main.py`	**Massive god-module** (~1450 lines). CLI, API, worker orchestration, upload, debug-clip, trim, reingest all in one file.	2
D2	HIGH	`backend/main.py` + `backend/api/job_worker.py`	**Optional module globals** (`db_instance`, `global_config`). NPE-prone; mypy-quarantined.	**1**
D3	HIGH	`backend/config/models.py`	**Dual config access.** Typed Pydantic model + legacy `.get()` dict shim. `extra='allow'` silently keeps typo'd keys.	**1**
D4	MEDIUM	`backend/config/loader.py`	Shallow top-level merge: a `config.json` section REPLACES the defaults dict for that section ? new fields missed on upgrade.	2
D5	MEDIUM	`backend/main.py`	`print()` used in production backend code (~20 calls in `--debug-clip`). Bypasses logging.	3
D6	MEDIUM	`backend/eval/`	Duplicate "annotations" modules: `backend.eval.annotations` vs `backend.annotations`. Different schemas, different error handling.	3
D7	MEDIUM	Multiple	Hardcoded tuning constants duplicated by copy (BASELINE, TUNED, padding).	3
D8	MEDIUM	`backend/pipeline/segmenters/base.py`	Registry instantiates `cls(None, {})` to read `.name` ? `__init__` must not touch `self.db`/`self.config`. Implicit contract.	3
D9	LOW	`backend/pipeline/validators/__init__.py`	Import mutates global registry as side-effect. Removing import = de-registering validator.	3

D10	LOW	`backend/reporting/spec.yaml` + `harvest.py`	Tight coupling via magic `when.path` strings. Rename a detail ? rule silently stops firing.	3
D11	LOW	`backend/pipeline/stitcher/compiler.py`	Two-stage codec contract is implicit. Change a preset ? concat breaks.	3
D12	LOW	`tools/generate_highlights.py`	Uses `sys.path.insert(0, ...)` hack instead of proper package install.	3
D13	MEDIUM	`requirements.txt`, `pyproject.toml`	**Absorbed from BACKLOG.** `supervision` ByteTrack is deprecated as of 0.28.0 and **removed in 0.30.0**. The `<0.30.0` pin in `requirements.txt` and `pyproject.toml` is a stopgap ? migration to the standalone `trackers` package is needed before the pin can be lifted.	2
D14	LOW	`tests/` directory	**Absorbed from BACKLOG.** 90+ test files in a flat `tests/` directory. Owner wants reorganization by subsystem (e.g. `tests/eval/`, `tests/audio/`) to reduce fragmentation. The exact grouping metric is the owner's design choice.	3

3c. Operational Risk

ID	Severity	Area	Issue	Phase
O1	HIGH	`backend/providers/gemini.py`	**Orphaned remote files on process death.** No GC for uploaded videos on Gemini File API.	2
O2	MEDIUM	`backend/main.py`	Single `ThreadPoolExecutor(max_workers=1)` ? one stuck job blocks all.	2
O3	MEDIUM	`backend/infrastructure/database.py`	SQLite WAL + busy_timeout but concurrent writers (CLI + server) risk `SQLITE_BUSY`.	3
O4	LOW	`backend/pipeline/detectors/tracknet_runner.py`	`weights_path=''` ? silent zero results. No startup weights-exist validation.	3
O5	LOW	`backend/utils/single_instance.py`	Windows byte-range locks may not work on network drives (Google Drive, SMB).	3

3d. Quick Wins (Low Risk, High Value)

ID	Area	Issue	Phase
Q1	`backend/config/models.py`	Fix the no-op `segmenter_must_be_registered` validator (same as B5).	**1**
Q2	`backend/pipeline/segmenters/motion.py`	Fix return type annotation: `List[Dict]` ? `Result[List[Dict[str, Any]]]`.	1
Q3	`run_eval.py`, `run_regression.py`, `scripts/`	Deduplicate `_default_db_path()` helper into a single shared utility.	1
Q4	`backend/config/loader.py`	Cache builtin defaults dict at module level (avoid double-instantiation of `AppConfig`).	1
Q5	`requirements.txt` vs `pyproject.toml`	Add comment explaining why `torch`/`torchvision` are in requirements.txt but not `pyproject.toml`.	1
Q6	`ruff.toml`	Enable `"I"` (isort) rule ? auto-fixable, consistent import ordering.	1
Q7	`mypy.ini`	Type-clean `motion.py` (small, legacy) as a ratchet exercise.	2
Q8	`backend/main.py`	Add comment explaining lazy `import cv2` in `--debug-clip` (cv2 not in LIGHT tier).	1

3e. Absorbed from BACKLOG

These findings were already recorded in `docs/BACKLOG.md` as deferred items from the 2026-06-10 hardening sweep. They are absorbed into this project for execution:

My ID	BACKLOG item	BACKLOG status	Absorbed as
D2	"App-factory / lifespan + lazy torch-cv2 imports"	? ? "Belongs with the async-job slice (#16)" ? #16 is now done; unblocked	Phase-1 target (P5)
O1	"WSL temp/frame/cache + Gemini remote-file GC"	? ? "staged videos, PNG frame dumps, `*_wasbcache`, and orphaned Gemini uploads are never cleaned"	Phase-2 target. The WSL half is partially handled by `keep_frames=false`; the Gemini half is new.
D13	"`supervision` ByteTrack deprecation"	? ? pinned `<0.30.0` as stopgap; migration to `trackers` package needed	New finding in this doc (see §3b)

| **D14** | "`tests/` reorganization to reduce fragmentation" | ? ? owner's design choice on grouping metric needed first | New finding in this doc (see §3b) |

Not absorbed (kept in BACKLOG as feature-gap, not code-cleanup):
- ? **Train/eval separation guardrails** ? `splits.py` exists but needs collector-side manifest change; this is a cross-repo feature, not a code-cleanup item.

---

4. Design / architecture ? Phase 1 approach

4a. D3 ? Typed-config migration (first target)

What: Convert all call sites that use `config.get("indexing",{}).get(...)` to typed `AppConfig` attribute access.

Reuse map:
- `backend/config/models.py` ? `AppConfig`, `IndexingConfig`, `ValidationConfig` (single source of truth for defaults + types)
- `backend/config/__init__.py` ? re-exports `load_config`, `AppConfig`
- `backend/pipeline/validators/*.py` ? 6 validators that consume `config: Dict[str, Any]`
- `backend/pipeline/segmenters/motion.py` ? uses `self.config.get("indexing",{})`
- `backend/reporting/__init__.py` ? already accepts both typed and dict; normalizes via `model_dump()`

Plan: Convert one module at a time, bottom-up. Validators first (they receive config as a dict, change the contract to `AppConfig`), then segmenters, then remove the `.get()` compat shim and flip to `extra='forbid'`.

Test strategy: Every module has existing tests. The config models have `test_config.py` that pins defaults. No new test behavior needed ? existing tests should pass byte-identically.

4b. B5 ? Segmenter validator fix (second target)

What: Make `segmenter_must_be_registered` actually validate against `segmenter_registry`.

File: `backend/config/models.py`, field validator on `IndexingConfig.default_segmenter`.

Risk: Near-zero. A typo'd segmenter in `config.json` that currently "works" (passes validation, fails at runtime) would now fail at startup. This is the DESIRED behavior ? catching config errors early.

4c. D2 ? App-factory refactor (third target, largest)

What: Replace `db_instance: Optional[Database] = None` + `global_config: Optional[AppConfig] = None` module globals with a `create_app(config) -> FastAPI` factory that injects state via `app.state`.

Reuse map:
- `backend/main.py` ? FastAPI app definition, all endpoints
- `backend/api/job_worker.py` ? resolves `db_instance`/`global_config` through `import backend.main as _main`
- `tests/test_api_endpoints.py` ? already uses `TestClient`; would switch to `create_app(test_config)`
- `tests/test_async_jobs.py` ? already monkeypatches `backend.main`

Plan: This is the highest-risk Phase-1 item but still a pure refactoring. The key insight: Starlette's `app.state` is the standard way to attach shared state; FastAPI's dependency injection can access it. Every endpoint that accesses `db_instance`/`global_config` would use `request.app.state.db` / `request.app.state.config` instead. The worker would receive these explicitly rather than resolving through the module object.

Test strategy: All existing tests must pass byte-identically. The `TestClient` usage changes from `from backend.main import app` to `from backend.main import create_app; app = create_app(test_config)`.

---

5. Honest limits ? what Phase 1 does NOT do

- **Does NOT change any behavior.** The Gemini provider still returns `[]` on failure; the motion segmenter still fabricates events; the PTS validator still has a hardcoded 10.0s gap.
- **Does NOT fix B2 (Success/Failure in Gemini).** That changes the return contract callers depend on ? deferred to Phase 2.
- **Does NOT split `main.py` (D1).** The god-module split is deferred ? D2 (app-factory) is a necessary precursor.
- **Does NOT address any operational risks (O1?O5).** Those need behavioral changes.
- **A green suite after Phase 1 does NOT mean the code is "clean."** It means the structural debt targeted by D2/D3/B5 is resolved. The remaining 28 items still need attention.

---

6. Deliverables & progress tracker ? **source of truth**

Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking.

ID	Deliverable	Depends on	Gated?	Status	PR
P0	**Docs-only PR ? this project doc** + index in `docs/README.md`	?	Owner approval	?	[#352](https://github.com/Khelsutra/badminton-highlight-indexer/pull/352)
?	**WASB cache test fix (Windows)** ? 2 pre-existing failures	?	No	?	[#353](https://github.com/Khelsutra/badminton-highlight-indexer/pull/353)
P4	**B5: Fix `segmenter_must_be_registered` validator**	P0	No	?	[#354](https://github.com/Khelsutra/badminton-highlight-indexer/pull/354)
P1	**D3-a: Convert validators to typed `AppConfig`** (6 files)	P0	No	?	[#355](https://github.com/Khelsutra/badminton-highlight-indexer/pull/355)
P6a	**Q3: Deduplicate `_default_db_path`**	P0	No	?	[#356](https://github.com/Khelsutra/badminton-highlight-indexer/pull/356)
P6b	**Q4: Cache builtin defaults**	P0	No	?	[#358](https://github.com/Khelsutra/badminton-highlight-indexer/pull/358)
P6c	**Q5: requirements.txt torch comment + Q6: isort (I) + Q8: cv2 comment**	P0	No	?	[#359](https://github.com/Khelsutra/badminton-highlight-indexer/pull/359) [#360](https://github.com/Khelsutra/badminton-highlight-indexer/pull/360) [#361](https://github.com/Khelsutra/badminton-highlight-indexer/pull/361)
T1	**Tier 1: Q2 (motion return type) + D11 (codec docstring)**	P0	No	?	[#362](https://github.com/Khelsutra/badminton-highlight-indexer/pull/362)
T2	**Tier 2: B6 (resolution_fps fallback + warning log)**	P0	No	?	[#363](https://github.com/Khelsutra/badminton-highlight-indexer/pull/363)
T3	**Tier 3+4: D12 (sys.path hack) + B3 (PTS gap config) + O5 (lock note) + D7 (centralize TUNED)**	P0	No	?	[#364](https://github.com/Khelsutra/badminton-highlight-indexer/pull/364)
P6d	~~D7: Centralize hardcoded tuning constants~~ ? folded into T3	P0	No	?	[#364](https://github.com/Khelsutra/badminton-highlight-indexer/pull/364)
P2	**D3-b: Convert segmenters to typed `AppConfig`** (15 files: 5 segmenters + 4 detectors + worker + eval + tools)	P1	No	?	[#366](https://github.com/Khelsutra/badminton-highlight-indexer/pull/366)
P3	**D3-c: Remove `.get()` compat shim; flip `extra='forbid'`** (18 files, +87/-169)	P2	No	?	[#367](https://github.com/Khelsutra/badminton-highlight-indexer/pull/367)
P5	**D2: App-factory refactor** ? replace module-level Optional globals with `create_app(config, db) ? FastAPI`	P3, P4	No	?	[#370](https://github.com/Khelsutra/badminton-highlight-indexer/pull/370)
P7	**Phase-1 verification:** full suite green, coverage ? 84%, mypy ratchet	P1?P6	No	?	1463/1463 pass (#371); `backend.api.job_worker` lifted from mypy quarantine
?	?	?	?	?	?

P8	**B2: Error-flagging in Gemini provider metrics** (Phase 2)	P7	No	?	[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373)
P9	**B1: Remove fabricated data from motion segmenter**	P7	No	?	?
P10	**D1: Split `main.py` god-module** (~1450 lines ? smaller modules)	P7	No	?	?
P11	**O1: Gemini remote-file GC** ? per-call `_safe_delete` (#373) + startup orphan sweep scoped to our `UPLOAD_TAG`-tagged files, grace-gated, HTTP-timeout-bounded	P7	No	?	[#380](https://github.com/Khelsutra/badminton-highlight-indexer/pull/380)
P12	**D13: ByteTrack deprecation** ? migrate to standalone `trackers` package	P7	No	?	?
P13	**D4: Deep config merge** ? section-level replace ? recursive merge	P7	No	?	[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373)
P14	**O2: Single-executor risk** ? documented with mitigations in job_worker.py	P7	No	?	[#374](https://github.com/Khelsutra/badminton-highlight-indexer/pull/374)
P15	**Q7: Type-clean `motion.py`** ? ratchet exercise	P7	No	?	[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373)
P17	**B2 (cont.): finish the `Success`/`Failure` return contract** ? the 4 callers that dropped provider error metrics (`trajectory_hybrid`, `yolo_hybrid`, `gemini_refine.refine_window`, `gemini_refine.full_video_rallies`) now surface `metrics["error"]` loudly instead of silently treating a failure as "0 rallies". (Full `Success`/`Failure` return-type migration of the hybrids is follow-up.)	P8	No	?	[#378](https://github.com/Khelsutra/badminton-highlight-indexer/pull/378)
P16+	Phase 3 items (B4/B7/B8/D5/D6/D8-D10/D14/O3/O4)	P15	No	?	?

---

7. Open questions

1. **Q: Should D1 (split `main.py`) be Phase 1 or Phase 2?** ? Leaning Phase 2: D2 (app-factory) is a natural precursor, and D1 is lower risk after the globals are gone. Owner to confirm.
2. ~~Q: Should Phase 1 include enabling `"I"` (isort) in ruff.toml?~~ ? ? Shipped in #360.

---

8. Definition of done

- [x] P0: This doc approved and merged to `master`
- [x] P1?P3: Typed-config migration complete ? no `.get()` dict access remains in backend code; `extra='forbid'` on `AppConfig`
- [x] P4: `segmenter_must_be_registered` actually validates against the registry
- [x] P5: `create_app()` injects shared state via `app.state` (handlers read `request.app.state`); `backend.api.job_worker` removed from mypy quarantine. (The Optional module globals `db_instance`/`global_config` are **retained as a transitional back-compat shim** and `backend.main` stays mypy-quarantined ? full removal is follow-up, see D1/P10.)
- [x] P6: Quick wins Q1?Q6, Q8 landed
- [x] P7: Full test suite ? 1463/1463 pass (#371); `ruff` + `mypy` clean; `backend.api.job_worker` lifted from quarantine
- [x] All PRs are single-purpose, branched from `master`, with descriptive commit messages
- [ ] `docs/README.md` updated to reflect the project's terminal state
- [ ] Final status flipped to `DONE` and this doc moved to `docs/archives/past_projects/`

8.1 ? Next items ? Phase 2 pick-up (prioritized)

All items below are **Claude-doable**, small-PR-friendly, and unblocked. Pick up in order:

Immediate (HIGH impact, low risk)

#	ID	What	Why

~~1~~	~~**B2 / P17**~~	~~**Finish** the `Success`/`Failure` return contract~~ ? **? DONE (this PR).** The 3 callers that dropped provider error metrics now surface `metrics["error"]` loudly. **Follow-up remains:** migrate the hybrid segmenters' `process_video` return type from bare `List[Dict]` to the ABC's `Result[...]` (`Success`/`Failure`) ? currently suppressed with `type: ignore[override]` in `trajectory_hybrid.py` (motion.py is already fully migrated).
~~2~~	~~**O1 / P11**~~	~~Gemini remote-file GC~~ ? **? DONE (#380).** Per-call `_safe_delete` (#373) + startup orphan sweep scoped to `UPLOAD_TAG`-tagged files, grace-gated, HTTP-timeout-bounded.

Near-term (MEDIUM impact)

#	ID	What	Why
3	**B1 / P9**	Remove fabricated data from `motion.py`	Random values for `server`/`receiver`/`shot_count`/`events`/`avg_speed_kmh` are persisted as real data. Replace with `None` or honest `"unknown"` markers.
4	**D13 / P12**	ByteTrack deprecation migration	`sv.ByteTrack` removed in supervision 0.30.0; pinned `<0.30.0` as stopgap. Migrate to standalone `trackers` package.
5	**D1 / P10**	Split `main.py` god-module	~1450 lines. D2 (app-factory) was the necessary precursor ? now done. Extract: CLI handlers ? `backend/cli/`, upload routes ? already in `backend/api/uploads.py`, debug helpers ? `scripts/`.

Later (deferred)

#	ID	What
6+	**P16+**	B4, B7, B8, D5, D6, D8?D10, D14, O3, O4 ? Phase 3 items

> **Removed from this list ? already shipped (see the §6 tracker); they were stale entries left from before Round 3/4 landed:** D4/P13 deep config merge (#373 ? `loader.py::_deep_merge`), O2/P14 single-executor risk documented (#374 ? `job_worker.py`), Q7/P15 `motion.py` type-cleaned (#373 ? no longer in the `mypy.ini` quarantine list).

---

9. References

- **Audit source:** 33 findings from a deep whole-repo read on 2026-06-24, plus 2 absorbed from `docs/BACKLOG.md` (D13, D14) for a total of 35. Two additional BACKLOG items (D2, O1) were already independently found and are cross-referenced in §3e.
- **Prior hardening sweep:** `docs/CODE_AUDIT_AND_TEST_HARDENING.md` (completed 2026-06-14) ? the previous audit-driven hardening round. This project is the next wave.
- **BACKLOG.md** `docs/BACKLOG.md` ? deferred items from the 2026-06-10 sweep; §3e maps the absorbed items.
- **CODE_MAP.md** `docs/CODE_MAP.md` ? placement rules, gotchas, and the cross-cutting footguns list.
- **mypy.ini** ? quarantine list and the ratchet instruction.
- **ruff.toml** ? lint rules; `"I"` (isort) deferred.
- **PROJECT_DOC_TEMPLATE.md** `docs/PROJECT_DOC_TEMPLATE.md` ? the template this doc follows.

Changelog
- `2026-06-24` ? Initial doc created from 33-item deep audit.
- `2026-06-24` ? Absorbed 2 findings from `docs/BACKLOG.md` (D13, D14). Cross-referenced D2 and O1 in §3e.
- `2026-06-24` ? Phase 1 begun. P0 (#352) ? #353 ? P4/B5 (#354) ? P1/D3-a (#355) ? P6a/Q3 (#356). Status ? IN PROGRESS.
- `2026-06-24` ? **Round 2.** Quick wins (#358 Q4, #359 Q5, #360 Q6 isort, #361 Q8) ? Tier 1 (#362 Q2+D11) ? Tier 2 (#363 B6) ? Tier 3+4 (#364 D12+B3+O5). 11 items shipped total.
- `2026-06-24` ? **Round 3.** P2/D3-b segmenters (#366) + P3/D3-c extra='forbid' (#367). 18 items shipped.

277   - `2026-06-24` ? **Round 4 (final).** P5/D2 app-factory (#370) + test fixes (#371).
Status ? PHASE 1 COMPLETE. Suite 1463/1463 green. §8.1 added with prioritized Phase 2 pick-up.
278   - `2026-06-25` ? **§8.1 reconciled with the §6 source-of-truth (docs-only).** Removed
three already-shipped items that §8.1 still listed as deferred future work ? D4/P13, O2/P14,
Q7/P15 (all merged in #373/#374) ? and split the unfinished half of B2 out of the done P8 into a
new **P17** (the provider return contract + the `trajectory_hybrid`/`gemini_refine` callers that
still drop `metrics["error"]`). Grounded against `loader.py::_deep_merge`, `mypy.ini`,
`backend/results.py`, and `backend/providers/gemini.py` + its callers. No code change.
279   - `2026-06-25` ? **Accuracy fixes (docs-only).** (1) §8 DoD P5 said the module globals
were "replaced with `app.state`"; corrected ? `create_app()` *added* `app.state` and lifted
`job_worker` from quarantine, but `db_instance`/`global_config` remain as a transitional shim
and `backend.main` is still mypy-quarantined (`backend/main.py:264`, `mypy.ini`). (2) `main.py`
size "~1300" ? "~1450" (now 1448 lines) in §3b/§6/§8.1.
280   - `2026-06-25` ? **P17 done (B2 finish).** The 4 callers that dropped provider error
metrics (`trajectory_hybrid.py`, `yolo_hybrid.py`, `gemini_refine.refine_window`,
`gemini_refine.full_video_rallies`) now check `metrics["error"]` and log loudly instead of
silently treating a provider failure as "0 rallies" ? matching the pattern the `gemini`
segmenter already uses. (`yolo_hybrid` was caught during PR #378 review ? it bound `metrics` but
never read it.) Suite 1484/1484 green; ruff + mypy clean. Follow-up: migrate the hybrids'
`process_video` return type from bare `List[Dict]` to the ABC `Result[...]` contract (motion.py
is already migrated; hybrids suppressed with `type: ignore[override]` for now).
281

7. assistant (2026-06-25T17:37:14.687Z)

The code change is correct (no `random` import, all values are `None`/honest markers). Now let me verify the key claims in the finding: that the convention was actually established and enforced by #380's review, and confirm the precedents (P8/#373, P11/#380, P17/#378) actually flipped the tracker in their own PRs. Let me check the git history of the audit doc to ground the "same-PR convention" claim.

8. user (2026-06-25T17:37:18.459Z)

```
===commits touching the audit doc===
cd5b3d2 feat(O1/P11): Gemini remote-file GC ? startup orphan sweep (#380)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)
f7fb36e docs(audit): fix two accuracy nits ? D2 DoD wording + main.py line count (#376)
e41cb59 docs(audit): reconcile §8.1 pick-up list with the §6 tracker (#375)
24178a7 docs(job_worker): document single-executor O2 risk + existing mitigations (#374)
d5a1fea fix: Phase 2 ? Q7 (motion type-clean) + B2 (Gemini error-flagging) + D4 (deep config
merge) (#373)
947a36b docs: update CODE_CLEANUP_AUDIT ? Phase 1 complete, Phase 2 prioritized (#372)
de09ec8 refactor(main): app-factory + typed-config migration (D2/D3/P2/P3/P5/P7) (#370)
801366e docs: update tracker ? D3-b (#366) + D3-c (#367) shipped; D2 (#368) planned (#369)
b50a6cc docs: update cleanup tracker ? round 2, 11 items shipped (#365)
bab72c6 docs: update cleanup project tracker ? Phase 1 items shipped (#357)
8e3aff1 docs: 2026-06-24 code-cleanup audit ? 33-item project doc (Phase 1 approval gate) (#352)

===Does PR #380 (commit cd5b3d2) touch the audit doc?===
    Refs: docs/CODE_CLEANUP_AUDIT_2026-06-24.md O1/P11.
    #3 Audit tracker: §6 P11 flipped to ? + #380 link; §8.1 #2 struck.
 docs/CODE_CLEANUP_AUDIT_2026-06-24.md |   4 +-

===Does PR #378 (P17) commit touch the audit doc?===
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)
bcd661f fix(B2/P17): surface Gemini provider failures in all callers
```

9. assistant (2026-06-25T17:37:24.726Z)

This is strong corroboration. The convention is real and consistently followed. Let me verify the precedents precisely - that #380 and #378 (code PRs, not docs-only PRs) flipped their own tracker rows in the same commit, and confirm the exact §6/§8.1/changelog edits they made.

10. user (2026-06-25T17:37:26.860Z)

===PR #380 (cd5b3d2) diff of audit doc ? code PR flipping its own tracker===

```
commit cd5b3d20c999b250ffcaf9f10c93473fa9b04704
Author: Avi <avi.dullu@gmail.com>
Date:   Thu Jun 25 22:53:51 2026 +0530

    feat(O1/P11): Gemini remote-file GC ? startup orphan sweep (#380)

    * feat(O1/P11): Gemini remote-file GC ? startup orphan sweep

    A hard process death (SIGKILL/OOM) between files.upload() and
    _safe_delete() orphans the uploaded clip on Google's servers forever.
    The per-call _safe_delete (already in place since #373) covers every
    normal/except path, but can't help when the process is killed mid-flight.

    This adds a best-effort startup sweep:
      - GeminiProvider.cleanup_orphaned_files(grace_sec=3600): lists all
        remote files, deletes any video/* older than the grace window (so a
        live worker mid-upload is never raced). Counts only successful
        deletes; swallows per-file + list failures (never blocks startup).
      - main._maybe_sweep_gemini_orphans(config): the startup hook ? a no-op
        unless ai_provider=='gemini' and GEMINI_API_KEY is set. Wired next
        to fail_stale_jobs() (the existing crash-recovery sweep).

    Tests: +6 (stale-video-only deletion, list-failure swallow, per-file-
    delete-failure swallow, skip-non-gemini, skip-no-key, error-swallow).
    Suite 1490/1490 green; mypy + ruff clean.

    Refs: docs/CODE_CLEANUP_AUDIT_2026-06-24.md O1/P11.

    * fix(O1/P11): address review ? scope sweep to our uploads, bound HTTP, datetime

    Review on #380 raised 3 MEDIUMs + LOWs; all addressed:

    #1 Shared-key blast radius: uploads now carry display_name=UPLOAD_TAG
       ('rally-indexer'); the sweep only deletes files matching that tag, so
       one instance can never delete another app's/account's files under a
       shared GEMINI_API_KEY.

    #2 Boot timeout: the client is now built with HttpOptions(timeout=30s)
       so a hung files.list()/delete() can't stall startup indefinitely
       (analyze_video's generate/poll were already bounded by ExecutionPolicy).

    #3 Audit tracker: §6 P11 flipped to ? + #380 link; §8.1 #2 struck.

    LOWs: create_time now handled via isinstance(datetime) (the SDK returns a
       tz-aware datetime, not an RFC3339 string ? the comment was wrong);
       +test for the real datetime path; +happy-path test deleting >1 file
       (off-by-one guard); mime filter dropped (the tag scope supersedes it);
       docstring claims softened to match reality.

    Suite 1492/1492 green; mypy + ruff clean.

    Refs: PR #380 review.

diff --git a/docs/CODE_CLEANUP_AUDIT_2026-06-24.md b/docs/CODE_CLEANUP_AUDIT_2026-06-24.md
index 87dfb6c..2393e32 100644
--- a/docs/CODE_CLEANUP_AUDIT_2026-06-24.md
+++ b/docs/CODE_CLEANUP_AUDIT_2026-06-24.md
@@ -196,7 +196,7 @@ Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One smal
 | **P8** | **B2: Error-flagging in Gemini provider metrics** (Phase 2) | P7 | No | ? |
[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
 | **P9** | **B1: Remove fabricated data from motion segmenter** | P7 | No | ? | ? |
 | **P10** | **D1: Split `main.py` god-module** (~1450 lines ? smaller modules) | P7 | No | ? |
? |
-| **P11** | **O1: Gemini remote-file GC** ? cleanup orphaned uploads on process death | P7 | No
```

```
| ? | ? |
+| **P11** | **O1: Gemini remote-file GC** ? per-call `_safe_delete` (#373) + startup orphan
sweep scoped to our `UPLOAD_TAG`-tagged files, grace-gated, HTTP-timeout-bounded | P7 | No | ? |
[#380](https://github.com/Khelsutra/badminton-highlight-indexer/pull/380) |
 | **P12** | **D13: ByteTrack deprecation** ? migrate to standalone `trackers` package | P7 | No
| ? | ? |
 | **P13** | **D4: Deep config merge** ? section-level replace ? recursive merge | P7 | No | ? |
[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
 | **P14** | **O2: Single-executor risk** ? documented with mitigations in job_worker.py | P7 |
No | ? | [#374](https://github.com/Khelsutra/badminton-highlight-indexer/pull/374) |
@@ -236,7 +236,7 @@ All items below are **Claude-doable**, small-PR-friendly, and unblocked.
Pick up
 | # | ID | What | Why |
 |---|----|------|-----|
 | ~~1~~ | ~~**B2 / P17**~~ | ~~**Finish** the `Success`/`Failure` return contract~~ ? **? DONE
(this PR).** The 3 callers that dropped provider error metrics now surface `metrics["error"]`
loudly. **Follow-up remains:** migrate the hybrid segmenters' `process_video` return type from
bare `List[Dict]` to the ABC's `Result[...]` (`Success`/`Failure`) ? currently suppressed with
`type: ignore[override]` in `trajectory_hybrid.py` (motion.py is already fully migrated). |
-| 2 | **O1 / P11** | Gemini remote-file GC | The per-call `_safe_delete` on every
failure/success path is ALREADY in place (`providers/gemini.py`). REMAINING: a cleanup-on-
startup sweep of orphaned files via `client.files.list()` for files left by a crashed previous
run. |
+| ~~2~~ | ~~**O1 / P11**~~ | ~~Gemini remote-file GC~~ ? **? DONE (#380).** Per-call
`_safe_delete` (#373) + startup orphan sweep scoped to `UPLOAD_TAG`-tagged files, grace-gated,
HTTP-timeout-bounded. |

 ### Near-term (MEDIUM impact)


========================================
===PR #378 (b72ac85) diff of audit doc===
commit b72ac85cf4a89311ebfa8a4e1b4b1ddf9b50e382
Author: Avi <avi.dullu@gmail.com>
Date:   Thu Jun 25 21:30:02 2026 +0530

    fix(B2/P17): surface Gemini provider failures in all callers (#378)

diff --git a/docs/CODE_CLEANUP_AUDIT_2026-06-24.md b/docs/CODE_CLEANUP_AUDIT_2026-06-24.md
index e537ec6..87dfb6c 100644
--- a/docs/CODE_CLEANUP_AUDIT_2026-06-24.md
+++ b/docs/CODE_CLEANUP_AUDIT_2026-06-24.md
@@ -201,7 +201,7 @@ Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One smal
 | **P13** | **D4: Deep config merge** ? section-level replace ? recursive merge | P7 | No | ? |
[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
 | **P14** | **O2: Single-executor risk** ? documented with mitigations in job_worker.py | P7 |
No | ? | [#374](https://github.com/Khelsutra/badminton-highlight-indexer/pull/374) |
 | **P15** | **Q7: Type-clean `motion.py`** ? ratchet exercise | P7 | No | ? |
[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
-| **P17** | **B2 (cont.): finish the `Success`/`Failure` return contract** ? P8/#373 added
error-flagging only; the provider still returns `[], metrics` on failure and
`trajectory_hybrid`/`gemini_refine` still drop it | P8 | No | ? | ? |
+| **P17** | **B2 (cont.): finish the `Success`/`Failure` return contract** ? the 4 callers that
dropped provider error metrics (`trajectory_hybrid`, `yolo_hybrid`,
`gemini_refine.refine_window`, `gemini_refine.full_video_rallies`) now surface
`metrics["error"]` loudly instead of silently treating a failure as "0 rallies". (Full
`Success`/`Failure` return-type migration of the hybrids is follow-up.) | P8 | No | ? |
[#378](https://github.com/Khelsutra/badminton-highlight-indexer/pull/378) |
 | **P16+** | Phase 3 items (B4/B7/B8/D5/D6/D8-D10/D14/O3/O4) | P15 | No | ? | ? |

 ---
@@ -235,8 +235,8 @@ All items below are **Claude-doable**, small-PR-friendly, and unblocked.
Pick up

 | # | ID | What | Why |
```

|---|----|------|-----|
-| 1 | **B2 / P17** | **Finish** the `Success`/`Failure` return contract on Gemini `analyze_video()` ? P8/#373 did only the flagging half | #373 added `metrics["error"]` on every failure path and the Gemini *segmenter* now logs it (`segmenters/gemini.py:230`, `:361`), but the provider still **returns `[], metrics` on failure** (`providers/gemini.py`) and two production callers still discard it (`trajectory_hybrid.py:329`, `gemini_refine.py:87`/`:176` use `segments, _ =`) ? so a provider failure is still reported as "0 rallies" there. The `Success`/`Failure` dataclasses exist in `backend/results.py`; wire them end-to-end and make those callers fail loudly. |
-| 2 | **O1 / P11** | Gemini remote-file GC | Uploaded files are never deleted on process death. Add `_safe_delete` in a `finally:` block around the generate call, plus a cleanup-on-startup sweep of orphaned files via `client.files.list()`. |
+| ~~1~~ | ~~**B2 / P17**~~ | ~~**Finish** the `Success`/`Failure` return contract~~ ? **? DONE (this PR).** The 3 callers that dropped provider error metrics now surface `metrics["error"]` loudly. **Follow-up remains:** migrate the hybrid segmenters' `process_video` return type from bare `List[Dict]` to the ABC's `Result[...]` (`Success`/`Failure`) ? currently suppressed with `type: ignore[override]` in `trajectory_hybrid.py` (motion.py is already fully migrated). |
+| 2 | **O1 / P11** | Gemini remote-file GC | The per-call `_safe_delete` on every failure/success path is ALREADY in place (`providers/gemini.py`). REMAINING: a cleanup-on-startup sweep of orphaned files via `client.files.list()` for files left by a crashed previous run. |

 ### Near-term (MEDIUM impact)

@@ -277,3 +277,4 @@ All items below are **Claude-doable**, small-PR-friendly, and unblocked. Pick up
 - `2026-06-24` ? **Round 4 (final).** P5/D2 app-factory (#370) + test fixes (#371). Status ? PHASE 1 COMPLETE. Suite 1463/1463 green. §8.1 added with prioritized Phase 2 pick-up.
 - `2026-06-25` ? **§8.1 reconciled with the §6 source-of-truth (docs-only).** Removed three already-shipped items that §8.1 still listed as deferred future work ? D4/P13, O2/P14, Q7/P15 (all merged in #373/#374) ? and split the unfinished half of B2 out of the done P8 into a new **P17** (the provider return contract + the `trajectory_hybrid`/`gemini_refine` callers that still drop `metrics["error"]`). Grounded against `loader.py::_deep_merge`, `mypy.ini`, `backend/results.py`, and `backend/providers/gemini.py` + its callers. No code change.
 - `2026-06-25` ? **Accuracy fixes (docs-only).** (1) §8 DoD P5 said the module globals were "replaced with `app.state`"; corrected ? `create_app()` *added* `app.state` and lifted `job_worker` from quarantine, but `db_instance`/`global_config` remain as a transitional shim and `backend.main` is still mypy-quarantined (`backend/main.py:264`, `mypy.ini`). (2) `main.py` size "~1300" ? "~1450" (now 1448 lines) in §3b/§6/§8.1.
+- `2026-06-25` ? **P17 done (B2 finish).** The 4 callers that dropped provider error metrics (`trajectory_hybrid.py`, `yolo_hybrid.py`, `gemini_refine.refine_window`, `gemini_refine.full_video_rallies`) now check `metrics["error"]` and log loudly instead of silently treating a provider failure as "0 rallies" ? matching the pattern the `gemini` segmenter already uses. (`yolo_hybrid` was caught during PR #378 review ? it bound `metrics` but never read it.) Suite 1484/1484 green; ruff + mypy clean. Follow-up: migrate the hybrids' `process_video` return type from bare `List[Dict]` to the ABC `Result[...]` contract (motion.py is already migrated; hybrids suppressed with `type: ignore[override]` for now).

11. assistant (2026-06-25T17:37:32.711Z)

The precedents are confirmed exactly as the finding claims. Both #380 (P11, a code PR) and #378 (P17, a code PR) flipped their own §6 row to ? with PR link, struck their §8.1 pick-up entry, and (#378) added a dated changelog entry ? all in the same PR as the code change. This is an established, consistently-followed convention.

Now let me confirm the exact stale spots the finding cites are present in the current doc state on this branch (the three concrete spots: line 197, line 245, and the missing changelog entry).

12. user (2026-06-25T17:37:37.303Z)

===Line 197 (§6 P9 row)===
| **P9** | **B1: Remove fabricated data from motion segmenter** | P7 | No | ? | ? |

===Line 245 (§8.1 B1/P9 pick-up)===
| 3 | **B1 / P9** | Remove fabricated data from `motion.py` | Random values for

`server`/`receiver`/`shot_count`/`events`/`avg_speed_kmh` are persisted as real data. Replace with `None` or honest `"unknown"` markers. |

===Last changelog entries===
- `2026-06-25` ? **§8.1 reconciled with the §6 source-of-truth (docs-only).** Removed three already-shipped items that §8.1 still listed as deferred future work ? D4/P13, O2/P14, Q7/P15 (all merged in #373/#374) ? and split the unfinished half of B2 out of the done P8 into a new **P17** (the provider return contract + the `trajectory_hybrid`/`gemini_refine` callers that still drop `metrics["error"]`). Grounded against `loader.py::_deep_merge`, `mypy.ini`, `backend/results.py`, and `backend/providers/gemini.py` + its callers. No code change.
- `2026-06-25` ? **Accuracy fixes (docs-only).** (1) §8 DoD P5 said the module globals were "replaced with `app.state`"; corrected ? `create_app()` *added* `app.state` and lifted `job_worker` from quarantine, but `db_instance`/`global_config` remain as a transitional shim and `backend.main` is still mypy-quarantined (`backend/main.py:264`, `mypy.ini`). (2) `main.py` size "~1300" ? "~1450" (now 1448 lines) in §3b/§6/§8.1.
- `2026-06-25` ? **P17 done (B2 finish).** The 4 callers that dropped provider error metrics (`trajectory_hybrid.py`, `yolo_hybrid.py`, `gemini_refine.refine_window`, `gemini_refine.full_video_rallies`) now check `metrics["error"]` and log loudly instead of silently treating a provider failure as "0 rallies" ? matching the pattern the `gemini` segmenter already uses. (`yolo_hybrid` was caught during PR #378 review ? it bound `metrics` but never read it.) Suite 1484/1484 green; ruff + mypy clean. Follow-up: migrate the hybrids' `process_video` return type from bare `List[Dict]` to the ABC `Result[...]` contract (motion.py is already migrated; hybrids suppressed with `type: ignore[override]` for now).

===Is the audit doc tracked + clean (not modified on this branch)?===
(empty above = unmodified)

===Confirm random import is gone from motion.py===
191:        breakdown ? those were previously fabricated with the stdlib ``random``

===Confirm no db.add_event calls remain===
NO add_event calls ? confirmed removed

13. assistant (2026-06-25T17:37:42.156Z)

All claims in the finding are verified. The only `random` reference in motion.py is in a docstring describing what was removed (historical context, not actual usage) - the code itself fabricates nothing. The three stale spots are confirmed present, the doc is unmodified on this branch, and the same-PR convention is established and consistently followed by precedent code PRs #380 and #378.

Let me make one final assessment on severity. The finding is a process/convention gap, not a code bug. The code is correct and CI is green. Let me weigh whether MEDIUM is the right severity given it's a documentation-tracker issue rather than a functional defect.

14. user (2026-06-25T17:37:52.709Z)

Structured output provided successfully

15. assistant (2026-06-25T17:37:58.016Z)

Verified the finding as REAL at MEDIUM severity. The PR completes B1/P9 in code (correctly ? `random` import removed, no `add_event` calls, all fabricated fields now `None`/honest markers) but omits the in-PR tracker edit that the project's established and consistently-enforced convention requires. I confirmed all three stale spots verbatim in `docs/CODE_CLEANUP_AUDIT_2026-06-24.md` and grounded the convention against the two immediately-preceding code PRs (#380 and #378), both of which flipped their own §6 row, struck their §8.1 pick-up entry, and (#378) added a dated changelog entry in the same PR as the code change.